

Task 75: K Closest Points to Origin

PROBLEM DESCRIPTION

Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane and an integer k , return the k closest points to the origin $(0, 0)$. The distance between two points on the X-Y plane is the Euclidean distance (i.e., $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$).

JAVA SOLUTION

```
import java.util.Arrays;
import java.util.Random;

class Solution {
    public int[][] kClosest(int[][] points, int k) {
        quickSelect(points, 0, points.length - 1, k);
        return Arrays.copyOfRange(points, 0, k);
    }

    private void quickSelect(int[][] points, int left, int right, int k) {
        if (left >= right) return;

        Random rand = new Random();
        int pivotIndex = left + rand.nextInt(right - left + 1);
        pivotIndex = partition(points, left, right, pivotIndex);

        if (k == pivotIndex) {
            return;
        } else if (k < pivotIndex) {
            quickSelect(points, left, pivotIndex - 1, k);
        } else {
            quickSelect(points, pivotIndex + 1, right, k);
        }
    }

    private int partition(int[][] points, int left, int right, int pivotIndex) {
        int pivotDist = dist(points[pivotIndex]);
        swap(points, pivotIndex, right);

        int storeIndex = left;
        for (int i = left; i < right; i++) {
            if (dist(points[i]) < pivotDist) {
                swap(points, storeIndex, i);
                storeIndex++;
            }
        }
        swap(points, storeIndex, right);
        return storeIndex;
    }

    private int dist(int[] point) {
        return point[0] * point[0] + point[1] * point[1];
    }
}
```

```
private void swap(int[][] points, int i, int j) {  
    int[] temp = points[i];  
    points[i] = points[j];  
    points[j] = temp;  
}  
}
```